Robust and Explainable Autoencoders for Unsupervised Time Series Outlier Detection

Tung Kieu[1], Bin Yang[1*], Chenjuan Guo[1], Christian S. Jensen[1], Yan Zhao[1], Feiteng Huang[2], Kai Zheng[3]
[1]Aalborg University, Denmark [2]Huawei Cloud Database Innovation Lab, China
[3]University of Electronic Science and Technology of China, China
{tungkvt, byang, cguo, csj, yanz}@cs.aau.dk, huangfeiteng@huawei.com, zhengkai@uestc.edu.cn

***Abstract*—Time series data occurs widely, and outlier detection is a fundamental problem in data mining, which has numerous applications. Existing autoencoder-based approaches deliver state-of-the-art performance on challenging real-world data but are vulnerable to outliers and exhibit low explainability. To address these two limitations, we propose robust and explainable unsupervised autoencoder frameworks that decompose an input time series into a clean time series and an outlier time series using autoencoders. Improved explainability is achieved because clean time series are better explained with easy-to-understand patterns such as trends and periodicities. We provide insight into this by means of a post-hoc explainability analysis and empirical studies. In addition, since outliers are separated from clean time series iteratively, our approach offers improved robustness to outliers, which in turn improves accuracy. We evaluate our approach on five real-world datasets and report improvements over the state-of-the-art approaches in terms of robustness and explainability.**

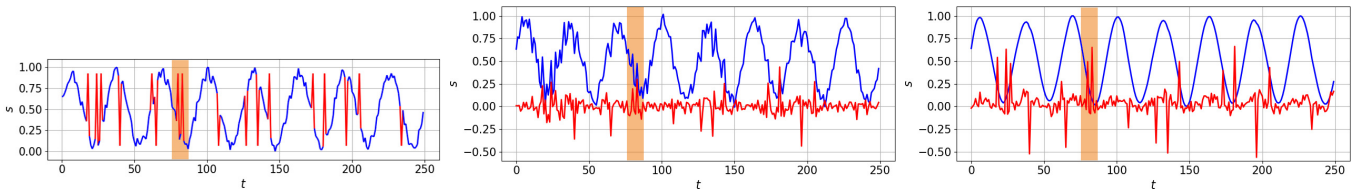
I. Introduction

Analyses of time series yield knowledge of the underlying processes that generate the time series and in turn enable us to understand those processes [1]–[5]. Time series outlier detection is a fundamental problem in data mining and has been applied in many applications, including in transportation, crowdsourcing, and manufacturing [6]–[10]. Recent neural network based methods for outlier detection in time series achieve strong performance on challenging datasets. These methods learn long-term, nonlinear temporal relationships in the data, outperforming existing, non-deep methods based on similarity search [11] and density-based clustering [12]. Neural network based methods first compress an input time series into a compact latent representation using an encoder and then reconstruct the original time series from the latent representation using a decoder. This encoder-decoder architecture, called an autoencoder [13], [14], imposes an information bottleneck [15] that encourages the compact latent representation to capture only the most representative patterns of the input time series while disregarding non-representative patterns such as outliers. This makes it possible to detect outliers by measuring the reconstruction error between the input time series and the reconstructed time series. The larger the reconstruction errors are, the more likely it is that the corresponding observations in the time series are outliers. This makes autoencoders fit unsupervised outlier detection, as they do not rely on any outlier labels that indicate which observations are outliers, but rely purely on reconstruction errors. Although autoencoders achieve competitive performance, they face two challenges.

(i) **Robustness:** In unsupervised settings, the training data may already include outliers. Since encoders compress all observations in an input time series, including outliers, the resulting latent representations are sensitive to outliers. Small numbers of outliers may still contaminate the latent representations, especially if their magnitudes are large. In other words, there is a risk that outliers in the training data pollute the latent representations such that the latent representations also capture the outlier patterns; thus, some outliers may have small reconstruction errors, which are then difficult to separate from clean data. This adversely affects accuracy. For example, the blue curve in Fig. 1b shows a reconstructed time series from a polluted latent representation. This yields small reconstruction errors for some outliers, making them difficult to detect (see the orange region in Fig. 1b). To avoid this, robust solutions are called for such that the latent representations are less affected by the outliers in the training data.

(ii) **Explainability:** Autoencoders regard observations with large reconstruction errors as outliers. Put differently, given an input time series $\mathcal{T}$, autoencoders regard the reconstructed time series $\hat{\mathcal{T}}$ as the "clean" time series that should occur if the underlying system generates the time series in a normal state. If observations from the input time series deviate substantially from the corresponding observations in the reconstructed time series, meaning that the corresponding reconstruction errors $\mathcal{T}-\hat{\mathcal{T}}$ are large, then the autoencoders regard these observation as outliers. Thus, to understand why particular observations are regarded as outliers by autoencoders, it is important to understand the corresponding reconstruction errors $\mathcal{T}-\hat{\mathcal{T}}$. Since the input time series $\mathcal{T}$ is given, it is then important to understand the key features of the reconstructed time series $\hat{\mathcal{T}}$, such as increasing or decreasing trends or different periodicities, as this offers information on which observations should occur in a normal state, which helps users understand the reconstruction errors $\mathcal{T}-\hat{\mathcal{T}}$ that indicate outliers. However, the reconstructed time series $\hat{\mathcal{T}}$ produced by existing autoencoders (e.g., the blue curve in Fig. 1b) are often complex with hard-to-explain specifics, making it difficult for users to understand which observations should occur in a normal state. This calls for more explainable solutions such that the reconstructed time series have a clear pattern and few hard-to-explain variations.

*Corresponding author.



(a) Input time series $\mathcal{T}$. Red spikes are outliers. (b) Standard autoencoder, RNNAE. (c) Robust autoencoder, RDAE.

Fig. 1: Robustness and Explainability Illustration. In (b) and (c), the blue curves indicate the reconstructed time series $\hat{\mathcal{T}}$ returned by RNNAE and RDAE, and the red curves indicate the reconstruction errors $\mathcal{T} - \hat{\mathcal{T}}$. *Robustness:* outliers generally have larger reconstruction errors in (c) than in (b), e.g., in the highlighted orange region, suggesting that RDAE is more robust to outliers. *Explainability:* the blue curve in (c) has a more clear periodic pattern and much less hard-to-explain variations than in (b), suggesting that RDAE has higher explainability than RNNAE.

We address these two limitations by providing two novel autoencoder frameworks that improve robustness and explainability. Instead of reconstructing an input time series $\mathcal{T}$ directly, we decompose it into two: a clean time series $\mathcal{T}_{\mathbf{L}}$ and an outlier time series $\mathcal{T}_{\mathbf{S}}$. We then employ autoencoders to reconstruct only the clean time series $\mathcal{T}_{\mathbf{L}}$, thus preventing outliers from polluting the latent representation. This improves robustness, which in turn improves accuracy. Fig. 1c shows the results of the proposed robust dual autoencoder RDAE, where the outliers in the orange region have larger reconstruction errors and are thus easier to detect.

Next, since the latent representation only captures clean time series, we expect our approach better explain the reconstructed time series in terms of trends and periodicities. For example, the blue curve in Fig. 1c shows the clean, reconstructed time series obtained from the proposed robust dual autoencoder RDAE. It includes clear periodic patterns without random variations, making it much easier for users to understand which observations are expected to occur in a normal state and hence understanding the reconstruction errors that indicate outliers, compared to that of Fig. 1b.

Finally, we propose a post-hoc explainability analysis method to quantify the explainability of autoencoder based outlier detection methods, which in turn facilitates quantitative comparisons of explainability among different autoencoder based outlier detection methods. Unlike in accuracy evaluation, where well-established metrics such as precision-recall exist, it lacks established metrics when evaluating the explainability of different autoencoder based outlier detection methods in a quantitative manner. Most studies evaluate explainability through visualizations (e.g., by contrasting Figs. 1b and 1c)—though intuitive, visualization based explainability evaluation fails to provide quantitative results. Motivated by explainable machine learning [16], we adopt post-hoc explainability analyses to design two metrics that enable quantitative evaluation of autoencoder based outlier detection methods.

To the best of our knowledge, this is the first study that focuses on improving both robustness and explainability of autoencoders for time series outlier detection. More specifically, we make three contributions. (i) We propose two autoencoder frameworks for unsupervised time series outlier detection that offer improved robustness and explainability. (ii) We propose a post-hoc explainability analysis technique that enables quantification of the explainability of autoencoder based outlier detection methods. (iii) We report on extensive empirical studies that offer insight into pertinent design properties of the two frameworks and that compare with baselines and the state-of-the-art approaches in terms of accuracy and explainability.

The rest of the paper is organized as follows. Section II covers preliminaries. Section III proposes the two frameworks. Section IV details the post-hoc explainability analysis. Section V reports experimental results, Section VI discusses related work, and Section VII concludes the paper.

II. Preliminaries

A. Time Series Outlier Detection

A time series $\mathcal{T} = \langle \mathbf{s}_1, \ldots, \mathbf{s}_C \rangle$ is a sequence of C observations, where each observation $\mathbf{s}_i \in \mathbb{R}^D$. If $D = 1$, $\mathcal{T}$ is *univariate*. If $D > 1$, $\mathcal{T}$ is *multivariate* (or *multidimensional*).

Given a time series $\mathcal{T} = \langle \mathbf{s}_1, \mathbf{s}_2, \ldots, \mathbf{s}_C \rangle$, we aim at computing an outlier score $\mathcal{OS}(\mathbf{s}_i)$ for each observation $\mathbf{s}_i$ such that the higher $\mathcal{OS}(\mathbf{s}_i)$ is, the more likely it is that observation $\mathbf{s}_i$ is an outlier. We make no assumptions whether outliers are point or collective outliers. If the outlier scores of continuous observations are high, these observations can be detected as a collective outlier.

B. Robust Principal Component Analysis

Given a matrix $\mathbf{M}$, Principal Component Analysis (PCA) is able to identify a low rank matrix to approximate matrix $\mathbf{M}$. Since PCA often employs Singular Value Decomposition (SVD) to identify the low-rank matrix, PCA also has the same problem as SVD of being very sensitive to outliers. To improve the performance of PCA when outliers exist, Robust Principal Component Analysis (RPCA) [17] has been proposed. RPCA aims to separate the given matrix $\mathbf{M}$ into the sum of two matrices—a low-rank matrix $\mathbf{L}$ that represents the clean data and a matrix $\mathbf{S}$ that consists of element-wise outliers. Specifically, RPCA decomposes the original matrix $\mathbf{M}$ such that $\mathbf{M} = \mathbf{L} + \mathbf{S}$. Here, $\mathbf{L}$ is a low-rank matrix that aims to approximate the clean data in the original matrix $\mathbf{X}$, and $\mathbf{S}$ is a sparse matrix that consists of element-wise outliers that should not be captured by the low-rank matrix $\mathbf{L}$. In other words, if the low rank matrix $\mathbf{L}$ tries to capture the outliers in $\mathbf{S}$ then $\mathbf{L}$ is skewed to the outliers and thus cannot capture appropriately the clean data. RPCA achieves the decomposition by solving the optimization problem shown in Eq. 1.

$$\arg\min_{\mathbf{L},\mathbf{S}} \text{rank}(\mathbf{L}) + \lambda ||\mathbf{S}||_0 \text{ s.t. } \mathbf{X} = \mathbf{L} + \mathbf{S} \tag{1}$$

Here, $\text{rank}(\mathbf{L})$ is the rank of matrix $\mathbf{L}$; $||\mathbf{S}||_0$ is the ℓ_0 norm of matrix $\mathbf{S}$, which counts the number of non-zero elements in $\mathbf{S}$; and λ is a coefficient that controls the relative importance of $||\mathbf{S}||_0$. In addition, the optimization is constrained by $\mathbf{M} = \mathbf{L}+\mathbf{S}$ because $\mathbf{M}$ is decomposed into $\mathbf{L}$ and $\mathbf{S}$. Minimizing the loss function makes it possible to identify a low rank matrix $\mathbf{L}$ that approximates the original matrix $\mathbf{M}$ and a sparse matrix $\mathbf{S}$ that includes outliers.

Although RPCA is able to effectively identify and remove outliers, it does not support time series directly. An additional limitation is that only linear transformations are employed.

C. Autoencoders

An autoencoder (AE) [13] consists of an encoder $E_{\theta_{AE}}(\cdot)$ and a decoder $D_{\theta_{AE}}(\cdot)$. The encoder $E_{\theta_{AE}}$ takes as input $\mathbf{M} \in \mathbb{R}^m$ and maps it into a compressed representation $\mathbf{H} \in \mathbb{R}^n$, where $n \ll m$. This occurs at the so-called *bottleneck layer*. Then, the decoder $D_{\theta_{AE}}$ takes $\mathbf{H}$ as input and outputs $\hat{\mathbf{M}} \in \mathbb{R}^m$, such that $\hat{\mathbf{M}}$ is as similar as possible to $\mathbf{M}$. This process is expressed in Eq. 2, and Fig. 2 offers a framework overview of AE.

$$\underset{\theta_{AE}}{\arg\min}\, ||\mathbf{M} - \hat{\mathbf{M}}||_2 = \underset{\theta_{AE}}{\arg\min}\, ||\mathbf{M} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{M}))||_2, \quad (2)$$

where θ_{AE} are the learnable parameters of the AE. When using neural networks, non-linear transformations can be added easily into the encoder and decoder, thus offering opportunities to solve the limitation in RPCA of only capturing linear relationships. In addition, there exist neural networks that capture well time series.

D. Design Considerations

We summarize the key design considerations underlying the different approaches in Table I. First, we consider whether a method supports time series data, noting that RPCA does not directly support time series data. Second, we consider whether a method is robust to outliers, which AEs are not because they work on all the data and do not distinguish outliers from clean data. Third, we consider explainability. AEs exhibit low explainability, while RPCA has strong theoretical underpinnings. Fourth, we consider whether a method is capable of supporting non-linear relationships, which often appear in complex time series. Here, RPCA falls short because it only uses linear operations and thus is only able to capture linear relationships. Fifth, we consider multi-view learning mechanism, which is not supported by any of RPCA or AEs. Based on the above analysis, we proceed to propose two robust and explainable autoencoder frameworks that achieve all the design considerations.

TABLE I: Summary of key design considerations.

	Time series	Robust	Explainable	Non-linear	Multi-view
RPCA	✗	✓	High	✗	✗
AE	✓	✗	Low	✓	✗
Ours	✓	✓	High	✓	✓

III. Methodology

A. Overall Idea

Although RPCA effectively identifies outliers, no attempt at using RPCA for time series outlier detection exist. Following the principles of RPCA, we propose neural net based AEs to decompose time series $\mathcal{T}$ into a clean time series $\mathcal{T}_{\mathbf{L}}$ and an outlier time series $\mathcal{T}_{\mathbf{S}}$ such that $\mathcal{T} = \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}$.

We expect $\mathcal{T}_{\mathbf{L}}$ to represent typical patterns of the underlying system that generates the time series, e.g., trends and periodicities. Then, the outlier time series $\mathcal{T}_{\mathbf{S}}$ is expected to include the data points that cannot be captured by the typical patterns. This offers explainability of the outlier detection process.

Since patterns in time series are often nonlinear, e.g., having different periodicities, we employ deep AEs with nonlinear activation functions. Specifically, we propose two AE frameworks that differ in how a time series is decomposed. The first framework employs a single AE to decompose the input time series $\mathcal{T}$ into clean and outlier parts. The second framework employs two AEs to decompose $\mathcal{T}$ from two different views. It first converts $\mathcal{T}$ into a lagged matrix $\mathbf{M}$. Then, the two AEs make the decomposition from the matrix vs. time series views, respectively. We proceed to elaborate the two frameworks.

B. Robust Autoencoders

We propose a *Robust Autoencoder* (RAE) that combines the benefits of an AE and RPCA, namely the non-linear computations and temporal dependencies of AEs and the robustness of RPCA (cf. Table I). The key insight is that the hidden representation in the bottleneck layer of an AE is very compact, and thus it is only possible to reconstruct representative features from the input data, but not the specifics of the input data such as outliers. Thus, outliers are not easily compressible and thus cannot be represented well in the bottleneck layer of an AE. Thus, we aim at obtaining a clean time series $\mathcal{T}_{\mathbf{L}}$ by removing an outlier time series $\mathcal{T}_{\mathbf{S}}$ from the original time series $\mathcal{T}$, i.e., $\mathcal{T}_{\mathbf{L}} = \mathcal{T} - \mathcal{T}_{\mathbf{S}}$. We then employ an AE to reconstruct only the clean time series $\mathcal{T}_{\mathbf{L}}$. Fig. 3 shows the overview of RAE framework. We formulate the following training objective:

$$\underset{\theta_{AE}, \mathcal{T}_{\mathbf{S}}}{\arg\min}\, ||\mathcal{T}_{\mathbf{L}} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}))||_2 + \lambda||\mathcal{T}_{\mathbf{S}}||_0 \text{ s.t. } \mathcal{T} = \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}. \quad (3)$$

The objective function of the AE in Eq. 3 has two terms. The first term measures the discrepancy between $\mathcal{T}_{\mathbf{L}}$ and its reconstruction $D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}))$. Since $\mathcal{T}_{\mathbf{L}}$ is supposed to have only clean data, an AE should be able to construct $\mathcal{T}_{\mathbf{L}}$ easily. The second term encourages the sparsity of outlier signals $\mathcal{T}_{\mathbf{S}}$ via ℓ_0 regularization, which is also used in RPCA [17]. Here, ℓ_0 counts the non-zero elements, which ensures the property of outliers that the number of outliers is small. Otherwise, they are not outliers, but instead represent some regular patterns [18]. Hyperparameter λ controls the relative importance of the regularization.

Optimizing Eq. 3 identifies not only the optimal parameters for the AE, θ_{AE}, but also the optimal outlier time series $\mathcal{T}_{\mathbf{S}}$. However, due to the constraint $\mathcal{T} = \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}$, traditional learning algorithms for neural networks, e.g., stochastic gradient descent, are not applicable. We design a bijective learning algorithm that alternates between (i) optimizing the first term

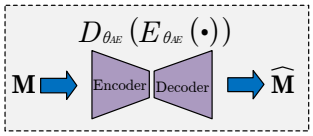


Fig. 2: AE framework.

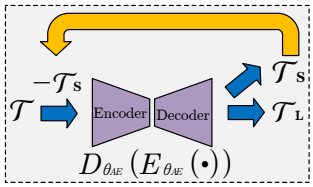


Fig. 3: RAE framework.

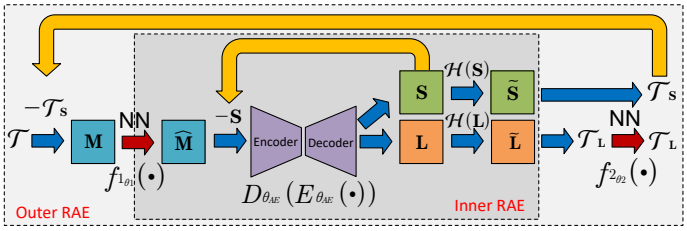


Fig. 4: RDAE framework.

to update θ_{AE} while keeping $\mathcal{T}_{\mathbf{S}}$ unchanged and (ii) optimizing the second term to update $\mathcal{T}_{\mathbf{S}}$ while keeping θ_{AE} unchanged. We ensure that the constraint is satisfied during the optimization. We detail the learning algorithm in Section III-F.

We choose a 1D convolutional neural network (1D CNN) as the computational units to build the AE because 1D CNNs have been shown to have an impressive performance on temporal data [19]. The 1D CNN-based RAE consists of two parts—an encoder $E_{\theta_{AE}}(\cdot)$ and a decoder $D_{\theta_{AE}}(\cdot)$, where each part consists of several 1D convolutional layers. The operations in the encoder $E_{\theta_{AE}}(\cdot)$ and the decoder $D_{\theta_{AE}}(\cdot)$ are shown in Eqs. 4 and 5.

$$E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_e * \mathcal{T}_{\mathbf{L}} + \mathbf{b}_e) \tag{4}$$

$$D_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_d * E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}) + \mathbf{b}_d) \tag{5}$$

Here, ϕ is a non-linear activation function, e.g., sigmoid, relu, or tanh. $\mathbf{W}_e$ and $\mathbf{b}_e$ are the weight matrix and bias vector of the encoder to create the feature maps in the encoder, respectively. $\mathbf{W}_d$ and $\mathbf{b}_d$ are the counterparts in the decoder. Generally, the number of feature maps of the encoder is less than the number of feature maps of the decoder to form a bottleneck layer. $*$ denotes the 1D convolution operator. In the encoding phase, following the convolutional layer, we use a max-pooling (i.e., down sampling method) layer to condense the data as well as increase the effect of having a bottleneck layer. In the decoding phase, in contrast to max-pooling, up-sampling is used to construct the output. The parameters in the AE include $\theta_{AE} = \{\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d\}$, which need to be learned.

Fig. 3 shows an overview of RAE. $\mathcal{T}_{\mathbf{S}}$, which is set to be empty in the first iteration, is subtracted from an input time series $\mathcal{T}$. The result is fed to the 1D CNN-based RAE. By solving the optimization problem shown in Eq. 3, $\mathcal{T}$ is split into two parts, $\mathcal{T}_{\mathbf{L}}$ and $\mathcal{T}_{\mathbf{S}}$. Among these, $\mathcal{T}_{\mathbf{S}}$ is used to update the input of the AE, i.e., $\mathcal{T} - \mathcal{T}_{\mathbf{S}}$, in the next iteration (cf. the yellow arrow in Fig. 3).

C. Robust Dual Autoencoders

Multi-view learning has been shown to be able to improve the robustness of learning algorithms by providing complementary information to the learner [20]. Intuitively, our framework could benefit from a multi-view formulation, e.g., a matrix view and a time series view. Motivated by this, we propose the second framework, *Robust Dual Autoencoders* (RDAE), which differs from RAE by using two robust AEs that help each other learn from two different representations of a time series, i.e., a matrix representation and a time series representation. RDAE also combines AEs and RPCA to leverage the benefits of both. Further, RDAE considers multi-view learning mechanism (cf. Table I).

Fig. 4 shows an overview of RDAE. We first embed $\mathcal{T} = \langle \mathbf{s}_1, \ldots, \mathbf{s}_C \rangle$ into a lagged matrix [21] $\mathbf{M} \in \mathbb{R}^{B \times K \times D}$. Here, we use a sliding window of size B, where $1 < B < C/2$, to iterate through $\mathcal{T}$ to obtain $K = C - B + 1$ columns that constitute $\mathbf{M}$ as follows. Recall that $\mathbf{s}_i$ in the lagged matrix is a D-dimensional vector. The anti-diagonal elements in matrix $\mathbf{M}$ are identical; such matrices are called *Hankel* matrices [22].

$$\mathbf{M} = \begin{bmatrix} \mathbf{s}_1 & \mathbf{s}_2 & \cdots & \mathbf{s}_K \\ \mathbf{s}_2 & \mathbf{s}_3 & \cdots & \mathbf{s}_{K+1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{s}_B & \mathbf{s}_{B+1} & \cdots & \mathbf{s}_C \end{bmatrix}$$

In RDAE, the time series is the first representation, whereas the lagged matrix is the second representation that reflects the geometry of a time series [23]. More specifically, the lagged matrix is able to represent shapes and patterns of time series such as the magnitude of trends and the frequency of periodicities [23]. Instead of decomposing $\mathcal{T}$ directly, we use an inner RAE to decompose matrix $\mathbf{M}$ into $\mathbf{L}$ and $\mathbf{S}$. Then, based on the two matrices, we use an outer RAE to decompose $\mathcal{T}$ into $\mathcal{T}_{\mathbf{L}}$ and $\mathcal{T}_{\mathbf{S}}$ (see Fig. 4).

Inner RAE: The inner RAE decomposes the lagged matrix $\mathbf{M}$; this can be viewed as a nonlinear version of RPCA. As shown in Fig. 4, rather than decomposing $\mathbf{M}$ directly, we first perform a *nonlinear transformation* of $\mathbf{M}$ by applying 2D CNN layers $\hat{\mathbf{M}} = f_{1_{\theta_1}}(\mathbf{M})$. We use 2D CNNs because the input is no longer a time series but a matrix. Specifically, we have:

$$\hat{\mathbf{M}} = f_{1_{\theta_1}}(\mathbf{M}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_1 * \mathbf{M} + \mathbf{b}_1) \tag{6}$$

Here, $\mathbf{W}_1, \mathbf{b}_1 \in \theta_1$ are the weight matrices and bias vectors of the *transformation*. The parameters in the transformation include $\theta_1 = \{\mathbf{W}_1, \mathbf{b}_1\}$, which need to be learned. The output $\hat{\mathbf{M}} \in \mathbb{R}^{B \times K \times D}$ has the same size as $\mathbf{M}$. A time series often contains small variations (i.e., noise) that is not viewed as outliers, but can still affect the accuracy of the model. The transformation has the effect of smoothing $\mathbf{M}$ by removing such noise. By doing this, we expect that the inner RAE achieves a cleaner decomposition, which enables a better reconstruction, thus improving robustness. We still want the smoothed matrix $\hat{\mathbf{M}}$ to be similar to $\mathbf{M}$. Thus, we design an objective function that minimizes the discrepancy between the two.

$$\arg\min_{\theta_1} ||\mathbf{M} - \hat{\mathbf{M}}||_2 = \arg\min_{\theta_1} ||\mathbf{M} - f_{1_{\theta_1}}(\mathbf{M})||_2 \tag{7}$$

After obtaining $\hat{\mathbf{M}}$, we employ an inner robust AE to decompose $\hat{\mathbf{M}}$ into $\mathbf{L}$ and $\mathbf{S}$. The AE has an encoder $E_{\theta_{AE}}$

and a decoder $D_{\theta_{AE}}$; both are based on 2D CNNs.

$$E_{\theta_{AE}}(\mathbf{L}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_e * \mathbf{L} + \mathbf{b}_e) \tag{8}$$

$$D_{\theta_{AE}}(\mathbf{L}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_d * E_{\theta_{AE}}(\mathbf{L}) + \mathbf{b}_d) \tag{9}$$

Here, $\mathbf{W}_e, \mathbf{b}_e \in \theta_{AE}$ are the weight matrices and bias vectors of the encoder, and $\mathbf{W}_d, \mathbf{b}_d \in \theta_{AE}$ are the the weight matrices and bias vectors of the decoder. The parameters in the encoder and the decoder include $\theta_{AE} = \{\mathbf{W}_e, \mathbf{b}_e, \mathbf{W}_d, \mathbf{b}_d\}$, which need to be learned.

We define a loss function for the inner RAE as follows.

$$\arg\min_{\theta_{AE}, \mathbf{S}} ||\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))||_2 + \lambda_1 ||\mathbf{S}||_0 \text{ s.t. } \hat{\mathbf{M}} = \mathbf{L} + \mathbf{S}. \tag{10}$$

The first term measures the discrepancy between $\mathbf{L}$ and the reconstructed $D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))$. The AE uses a bottleneck layer to reconstruct $\mathbf{L}$, which mimics the way RPCA uses a low-rank matrix. Second, it employs an ℓ_0 regularization to $\mathcal{T}_\mathbf{S}$ following the regularization term of RPCA. Finally, the optimization is constrained by $\hat{\mathbf{M}} = \mathbf{L} + \mathbf{S}$.

Outer RAE: The outer RAE decomposes $\mathcal{T}$ into $\mathcal{T}_\mathbf{L}$ and $\mathcal{T}_\mathbf{S}$ while considering the output matrices $\mathbf{L}$ and $\mathbf{S}$ from the inner RAE. Specifically, we first employ the Hankelization operator [22] to transform matrices $\mathbf{L}$ and $\mathbf{S}$ into Hankel matrices: $\tilde{\mathbf{L}} = \mathcal{H}(\mathbf{L})$ and $\tilde{\mathbf{S}} = \mathcal{H}(\mathbf{S})$. Next, we covert the Hankel matrices back to time series $\mathcal{T}_\mathbf{L}$ an $\mathcal{T}_\mathbf{S}$ using the reverse of the operation that embeds a time series into a lagged matrix. Then we feed $\mathcal{T}_\mathbf{L}$ to 1D CNN layers, $f_{2_{\theta_2}}(\cdot)$, to perform a nonlinear transformation. The output is an updated clean time series with the same size. Specifically, we have:

$$\mathcal{T}_\mathbf{L} = f_{2_{\theta_2}}(\mathcal{T}_\mathbf{L}) \stackrel{\text{def}}{=} \phi(\mathbf{W}_2 * \mathcal{T}_\mathbf{L} + \mathbf{b}_2) \tag{11}$$

Here, $\mathbf{W}_2, \mathbf{b}_2 \in \theta_2$ are the weight matrices and bias vectors of the outer RAE. The parameters in the outer RAE include $\theta_2 = \{\mathbf{W}_2, \mathbf{b}_2\}$, which need to be learned. While doing this, we utilize the 1D CNN layers as the outer RAE by solving the following optimization problem.

$$\arg\min_{\theta_2, \mathcal{T}_\mathbf{S}} ||\mathcal{T}_\mathbf{L} - f_{2_{\theta_2}}(\mathcal{T}_\mathbf{L})||_2 + \lambda_2 ||\mathcal{T}_\mathbf{S}||_0 \text{ s.t. } \mathcal{T} = \mathcal{T}_\mathbf{L} + \mathcal{T}_\mathbf{S} \tag{12}$$

This loss function (i) minimizes the loss between time series $\mathcal{T}_\mathbf{L}$, which is derived from the clean matrix $\mathbf{L}$, and the reconstructed time series $f_{2_{\theta_2}}(\mathcal{T}_\mathbf{L})$; and (ii) it causes $\mathcal{T}_\mathbf{S}$ to be sparse due to the ℓ_0 regularization. The optimization is constrained by $\mathcal{T} = \mathcal{T}_\mathbf{L} + \mathcal{T}_\mathbf{S}$ as the outer RAE decomposes the time series $\mathcal{T}$.

Interplay between the Inner and Outer RAEs: The two AEs help each other achieve the best separation of outliers using clean data from different representations. The first is based on the lagged matrix, while the second works on the time series while taking into account the lagged matrix decomposition. Having obtained $\mathcal{T}_\mathbf{L}$ from the outer RAE, we use $\mathcal{T}_\mathbf{L}$ as a new time series $\mathcal{T}$ such that the process continues iteratively until convergence.

D. Outlier Scoring

Following existing studies [19], [24], we assign an outlier score to each observation in time series $\mathcal{T}$. Since we decomposed $\mathcal{T}$ into a clean time series $\mathcal{T}_\mathbf{L}$ and an outlier time series $\mathcal{T}_\mathbf{S}$, the outlier scores $\mathcal{OS}$ are the norm values of observations in $\mathcal{T}_\mathbf{S} = \langle \mathbf{s}_{\mathbf{S}_1}, \mathbf{s}_{\mathbf{S}_2}, \ldots, \mathbf{s}_{\mathbf{S}_C} \rangle$: $\mathcal{OS} = \langle ||\mathbf{s}_{\mathbf{S}_1}||_2^2, ||\mathbf{s}_{\mathbf{S}_2}||_2^2, \ldots, ||\mathbf{s}_{\mathbf{S}_C}||_2^2 \rangle$. We use the ℓ_2 norm to convert vectors to scalar values that are more intuitive.

E. Reformulating the Loss Functions

The loss functions of both RAE and RDAE include a term using the ℓ_0 norm that aims to make the outlier matrix or outlier time series sparse, thus ensuring the semantics of outliers. However, the ℓ_0 norm is non-convex, which makes optimization difficult [25]. Following the literature [26], we therefore relax the ℓ_0 norm to the ℓ_1 norm that is a good approximation of the ℓ_0 norm. The reformulated loss function of RAE is shown in Eq. 13.

$$\arg\min_{\theta_{AE}, \mathcal{T}_\mathbf{S}} ||\mathcal{T}_\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_\mathbf{L}))||_2 + \lambda ||\mathcal{T}_\mathbf{S}||_1 \text{ s.t. } \mathcal{T} = \mathcal{T}_\mathbf{L} + \mathcal{T}_\mathbf{S} \tag{13}$$

The reformulated loss functions of RDAE are shown in Eqs. 14, 15, and 16.

$$\begin{cases} \arg\min_{\theta_1} ||\mathbf{M} - f_{1_{\theta_1}}(\mathbf{M})||_2 & (14) \\ \arg\min_{\theta_{AE}, \mathbf{S}} ||\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))||_2 + \lambda_1 ||\mathbf{S}||_1 \text{ s.t. } \hat{\mathbf{M}} = \mathbf{L} + \mathbf{S} & (15) \\ \arg\min_{\theta_2, \mathcal{T}_\mathbf{S}} ||\mathcal{T}_\mathbf{L} - f_{2_{\theta_2}}(\mathcal{T}_\mathbf{L})||_2 + \lambda_2 ||\mathcal{T}_\mathbf{S}||_1 \text{ s.t. } \mathcal{T} = \mathcal{T}_\mathbf{L} + \mathcal{T}_\mathbf{S} & (16) \end{cases}$$

F. Training Algorithms

The optimization problems of RAE and RDAE have constraints and thus cannot be solved by gradient descent based back-propagation (BACKPROP). Our optimization problems are two-block optimization problems, which can be solved by the *Alternating Direction Method of Multipliers* (ADMM) [27]. The core idea of ADMM is to divide an objective function into multiple sub-objectives. Then, ADMM optimizes one sub-objective function while keeping the remaining sub-objectives fixed. After optimizing a sub-objective function, the constraint is enforced. In addition, we use the *Proximal Algorithm* (PROX) [28] to optimize elements inside the ℓ_1 norm.

Optimizing RAE (Algorithm 1): We split the reformulated objective function of RAE (see Eq. 13) into two parts. The first part optimizes θ_{AE} to minimize $||\mathcal{T}_\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_\mathbf{L}))||_2$; the second part optimizes $\mathcal{T}_\mathbf{S}$ to minimize $\lambda ||\mathcal{T}_\mathbf{S}||_1$. Finally, we update $\mathcal{T}_\mathbf{L}$ to be $\mathcal{T} - \mathcal{T}_\mathbf{S}$ to enforce the constraint. The result is then fed into the next iteration (see the yellow arrow in Fig. 3). The first stopping condition, *condition*$_1$, ends the optimization when it satisfies the constraints $\mathcal{T} = \mathcal{T}_\mathbf{L} + \mathcal{T}_\mathbf{S}$. The second stopping condition, *condition*$_2$, ends the optimization when $\mathcal{T}_\mathbf{L}$ and $\mathcal{T}_\mathbf{S}$ are fixed.

Optimizing RDAE (Algorithm 2): We use a loop to control the overall flow. Inside the loop, we optimize each reformulated objective function of RDAE in turn (see Eqs. 14, 15, and 16). For

Algorithm 1: Training algorithm for RAE

Input: Time series $\mathcal{T}$, double λ, double ϵ
Output: $\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{\mathbf{S}}$
Initialization: $\mathcal{T}_{\mathbf{L}} \leftarrow 0$; $\mathcal{T}_{\mathbf{S}} \leftarrow 0$; $\mathcal{T}^* \leftarrow \mathcal{T}$;
repeat
  $\mathcal{T}_{\mathbf{L}} \leftarrow \mathcal{T} - \mathcal{T}_{\mathbf{S}}$;
  // Optimize AE θ_{AE}
  Update θ_{AE} by minimizing $||\mathcal{T}_{\mathbf{L}} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}))||_2$ using BACKPROP;
  $\mathcal{T}_{\mathbf{L}} \leftarrow D_{\theta_{AE}}(E_{\theta_{AE}}(\mathcal{T}_{\mathbf{L}}))$;
  $\mathcal{T}_{\mathbf{S}} \leftarrow \mathcal{T} - \mathcal{T}_{\mathbf{L}}$;
  // Optimize $\mathcal{T}_{\mathbf{S}}$
  Update $\mathcal{T}_{\mathbf{S}}$ by minimizing $\lambda||\mathcal{T}_{\mathbf{S}}||_1$ using PROX;
  // Compute stopping conditions
  $condition_1 \leftarrow ||\mathcal{T} - \mathcal{T}_{\mathbf{L}} - \mathcal{T}_{\mathbf{S}}||_2/||\mathcal{T}||_2$;
  $condition_2 \leftarrow ||\mathcal{T}^* - \mathcal{T}_{\mathbf{L}} - \mathcal{T}_{\mathbf{S}}||_2/||\mathcal{T}||_2$;
  $\mathcal{T}^* \leftarrow \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}$;
until $condition_1 < \epsilon$ *or* $condition_2 < \epsilon$;
return $\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{\mathbf{S}}$;

Algorithm 2: Training algorithm for RDAE

Input: Time series $\mathcal{T}$, window length B, double λ_1, double λ_2, double ϵ
Output: $\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{\mathbf{S}}$
1 Initialization: $\mathbf{L} \leftarrow 0$; $\mathbf{S} \leftarrow 0$; $\mathcal{T}_{\mathbf{L}} \leftarrow 0$; $\mathcal{T}_{\mathbf{S}} \leftarrow 0$;
2 **while** *not converged* **do**
3 $\mathcal{T}_{\mathbf{L}} \leftarrow \mathcal{T} - \mathcal{T}_{\mathbf{S}}$;
4 Create lagged matrix $\mathbf{M} \in \mathbb{R}^{B \times K \times D}$ from $\mathcal{T}_{\mathbf{L}}$;
5 Update θ_1 by minimizing $||\mathbf{M} - f_{1_{\theta_1}}(\mathbf{M})||_2$ using BACKPROP;
6 $\hat{\mathbf{M}} \leftarrow f_{1_{\theta_1}}(\mathbf{M})$;
7 $\hat{\mathbf{M}}^* \leftarrow \hat{\mathbf{M}}$;
8 **repeat**
9 $\mathbf{L} \leftarrow \hat{\mathbf{M}} - \mathbf{S}$;
  // Optimize inner AE θ_{AE}
10 Update θ_{AE} by minimizing $||\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))||_2$ using BACKPROP;
11 $\mathbf{L} \leftarrow D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))$;
12 $\mathbf{S} \leftarrow \hat{\mathbf{M}} - \mathbf{L}$;
  // Optimize $\mathbf{S}$
13 Update $\mathbf{S}$ by minimizing $\lambda_1||\mathbf{S}||_1$ using PROX;
  // Compute stopping conditions
14 $condition_1 \leftarrow ||\hat{\mathbf{M}} - \mathbf{L} - \mathbf{S}||_2/||\hat{\mathbf{M}}||_2$;
15 $condition_2 \leftarrow ||\hat{\mathbf{M}}^* - \mathbf{L} - \mathbf{S}||_2/||\hat{\mathbf{M}}||_2$;
16 $\hat{\mathbf{M}}^* \leftarrow \mathbf{L} + \mathbf{S}$;
17 **until** $condition_1 < \epsilon$ *or* $condition_2 < \epsilon$;
18 Create Hankel matrices $\tilde{\mathbf{L}} \leftarrow \mathcal{H}(\mathbf{L})$ and $\tilde{\mathbf{S}} \leftarrow \mathcal{H}(\mathbf{S})$;
19 Obtain time series $\mathcal{T}_{\mathbf{L}}$ and $\mathcal{T}_{\mathbf{S}}$ from the two Hankel matrices;
20 Initialization: $\mathcal{T}^* \leftarrow \mathcal{T}$;
21 **repeat**
22 $\mathcal{T}_{\mathbf{L}} \leftarrow \mathcal{T} - \mathcal{T}_{\mathbf{S}}$;
  // Optimize outer AE θ_2
23 Update θ_2 by minimizing $||\mathcal{T}_{\mathbf{L}} - f_{2_{\theta_2}}(\mathcal{T}_{\mathbf{L}})||_2$ using BACKPROP;
24 $\mathcal{T}_{\mathbf{L}} \leftarrow f_{2_{\theta_2}}(\mathcal{T}_{\mathbf{L}})$;
25 $\mathcal{T}_{\mathbf{S}} \leftarrow \mathcal{T} - \mathcal{T}_{\mathbf{L}}$;
  // Optimize $\mathcal{T}_{\mathbf{S}}$
26 Update $\mathcal{T}_{\mathbf{S}}$ by minimizing $\lambda_2||\mathcal{T}_{\mathbf{S}}||_1$ using PROX;
  // Compute stopping conditions
27 $condition_1 \leftarrow ||\mathcal{T} - \mathcal{T}_{\mathbf{L}} - \mathcal{T}_{\mathbf{S}}||_2/||\mathcal{T}||_2$;
28 $condition_2 \leftarrow ||\mathcal{T}^* - \mathcal{T}_{\mathbf{L}} - \mathcal{T}_{\mathbf{S}}||_2/||\mathcal{T}||_2$;
29 $\mathcal{T}^* \leftarrow \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}$;
30 **until** $condition_1 < \epsilon$ *or* $condition_2 < \epsilon$;
31 **return** $\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{\mathbf{S}}$;

Eq. 14, we optimize for θ_1 to minimize $||\mathbf{M} - f_{1_{\theta_1}}(\mathbf{M})||_2$ using BACKPROP. For Eq. 15, we split the optimization problem into two parts. The first part trains the inner RAE by finding an optimal θ_{AE} to minimize $||\mathbf{L} - D_{\theta_{AE}}(E_{\theta_{AE}}(\mathbf{L}))||_2$; the second part finds an optimal $\mathbf{S}$ to minimize $\lambda_1||\mathbf{S}||_1$. We also need to ensure that the constraint $\hat{\mathbf{M}} = \mathbf{L} + \mathbf{S}$ is satisfied. For Eq. 16, we also split the optimization problem into two parts. The first part trains the outer RAE by finding an optimal θ_2 to minimize $||\mathcal{T}_{\mathbf{L}} - f_{2_{\theta_2}}(\mathcal{T}_{\mathbf{L}})||_2$; the second part finds an optimal $\mathcal{T}_{\mathbf{S}}$ to minimize $\lambda_2||\mathcal{T}_{\mathbf{S}}||_1$. To connect the inner RAE and the outer RAE, which are trained by ADMM, we subtract $\mathcal{T}_{\mathbf{S}}$ from $\mathcal{T}$ to update $\mathbf{M}$ and $\hat{\mathbf{M}}$ (lines 4 and 6). Then, $\mathbf{S}$ is subtracted from $\hat{\mathbf{M}}$ (line 9) to update $\mathbf{L}$ (line 11) and $\mathbf{S}$ (line 13). From the updated $\mathbf{L}$ and $\mathbf{S}$ (line 18), $\mathcal{T}_{\mathbf{L}}$ and $\mathcal{T}_{\mathbf{s}}$ are updated (lines 24 and 26). When subtracting $\mathcal{T}_{\mathbf{S}}$ from $\mathcal{T}$ in the first few iterations, $\mathcal{T}_{\mathbf{S}}$ at that time has not yet converged. Then, the $\hat{\mathbf{M}}$ has not converged either. Thus, $\mathbf{S}$ must be subtracted from $\hat{\mathbf{M}}$. The first stopping condition in the inner RAE and in the outer RAE, $condition_1$, ends the optimization when it satisfies the constraint $\hat{\mathbf{M}} = \mathbf{L} + \mathbf{S}$ and the constraints $\mathcal{T} = \mathcal{T}_{\mathbf{L}} + \mathcal{T}_{\mathbf{S}}$, respectively. The second stopping condition in the inner RAE and in the outer RAE, $condition_2$, ends the optimization when $\mathbf{L}$ and $\mathbf{S}$ are fixed and when $\mathcal{T}_{\mathbf{L}}$ and $\mathcal{T}_{\mathbf{S}}$ are fixed, respectively.

Convergence Analysis: Although the objective functions used in RAE and RDAE are not convex, following existing studies [29], [30], optimizing them using ADMM ensures convergence [29], [30]. In addition, we provide empirical evidence for the convergence of the proposed algorithms in Section V-B.

Complexity Analysis: The complexity of RAE is $O(e \cdot C \cdot D \cdot N \cdot L)$, where e is the number of epochs, C is the time series length, D is the time series dimensionality, N is the number of kernels, and L is number of convolutional layers. The complexity of RDAE is the sum of the complexities of the transformation, the inner RAE, and the outer RAE. Here, the inner RAE's complexity dominates the other complexities because the inner RAE works on the lagged matrix that is much larger than the original time series. The complexity of the inner RAE is $O(e \cdot B \cdot K \cdot D \cdot N \cdot L)$, where B is the window size and $K = C - B + 1$.

IV. Explainability Analysis

A. Concept and Motivation

Autoencoder based outlier detection methods regard observations with large reconstruction errors, i.e., large deviations from the "clean" time series, as outliers. In traditional AEs, the reconstructed time series are considered as the clean time series. In the two proposed robust autoencoder frameworks, the decomposed clean time series $\mathcal{T}_{\mathbf{L}}$ is the clean time series. The clean time series represents the observations that should occur if the underlying system generating the time series is in a normal state; and the observations with large deviations w.r.t. the corresponding observations from the clean time series, i.e., with large reconstruction errors, are outliers. Thus, to explain why an autoencoder based method regards some observations as outliers, it is important to understand the reconstruction errors $\mathcal{T} - \mathcal{T}_{\mathbf{L}}$. Since the input time series $\mathcal{T}$ is given, it is then important that the users understand the key features of the clean time series $\mathcal{T}_{\mathbf{L}}$ returned by the autoencoder method, e.g., increasing or decreasing trends, and different periodicities. This helps users know which observations should occur in a normal state, which then facilitates the users to understand the reconstruction errors and the identified outliers.

Based on the above intuition, we propose two explainability scores to quantify the explainability of different autoencoder based outlier detection methods by measuring how easily a user can understand the key features of the clean time series (e.g., trend and seasonality) returned by such methods. We expect a framework to offer high explainability, if its derived clean time series includes easy-to-understand patterns (e.g., trend and seasonality). Then a user can easily grasp the key features of the clean time series and understand which observations should occur at particular timestamps. The easier a user can understand the patterns in the clean time series, the higher the explainability of a framework.

Fig. 5 illustrates the intuition. It shows an input time series $\mathcal{T}$ with three outliers as well as three autoencoder methods A, B, and C that each derives a clean time series (in blue) to represent the input time series, denoted by $\mathcal{T}_{\mathbf{L}}^{A}$, $\mathcal{T}_{\mathbf{L}}^{B}$, and $\mathcal{T}_{\mathbf{L}}^{C}$, respectively. We consider framework A to be more explainable than framework B because $\mathcal{T}_{\mathbf{L}}^{A}$ captures the monotonically increasing trend and the periodicities of the input time series. Users can easily grasp the key features of the clean time series and thus understand which observations should occur at different timestamps. In contrast, the clean time series $\mathcal{T}_{\mathbf{L}}^{B}$ produced by framework B is more complex and includes arbitrary variations that cannot easily be attributed to the underlying process being sampled, thus making it difficult for a user to understand the observations at particular timestamps.

If we want to represent the clean time series derived by methods A and B by functions, we have to use a more complex function to represent accurately time series $\mathcal{T}_{\mathbf{L}}^{B}$ than to represent time series $\mathcal{T}_{\mathbf{L}}^{A}$. This indicates that framework B has a lower explainbility than framework A. We propose two explainability metrics that build on Polynomial Regression Models (PRM) [31] and Singular Spectrum Analysis (SSA) [21] to enable quantification of explainablity.

Finally, framework C in Fig. 5 is even more explainable than framework A—time series $\mathcal{T}_{\mathbf{L}}^{C}$ captures only the monotonically increasing trend of the input time series and thus can be described accurately by an even simpler function. However, this occurs at the cost of poor *outlier detection accuracy*, as many observations have large outlier scores and are thus mistakenly considered as outliers. We include the third framework C to make the point that a solution cannot just produce very simple clean time series that give high explainability—methods are constrained by also having to be accurate in terms of outlier detection. In the experiments, we show that the two proposed robust methods achieve high explainability while maintaining very high accuracy.

The proposed explainability analysis along with the corresponding quantification method differs from identifying the "root causes" of outliers [11], [32], [33]. The root cause of outliers are application/domain specific and indicate the reasons for malfunctions in the underlying system that generates the time series [11]. For example, a root cause can be an unexpected behavior (e.g., a network flooding attack in server-load time series), an accidental event (e.g., a car accident in traffic flow time series), an external impact (e.g., an international conflict causes a dramatic increase in energy prices), or a system failure (e.g., a short circuit in time series that monitor an electrical system). Identifying such root causes relies on supervised learning methods, e.g., classification methods that assign predefined labels that associate different root causes with identified outliers. This requires human experts to annotate manually a large amount of observations, indicating both which observations are outliers and the corresponding root causes of the outliers.

A few application/domain-agnostic root cause analysis methods exist [33], which work in unsupervised settings. Such methods identify the most anomalous channel for each detected outlier observation. For example, a peak in a CPU-load channel can cause an outlier in a 3-dimensional server-load time series that consists of CPU-, RAM-, and Network-load channels. Here, the CPU-load channel is considered as the root cause for the outlier.

In summary, the existing root cause identification methods try to explain why specific observations are identified as outliers. Unlike the existing methods, we aim at quantifying the explainability of autoencoder based outlier detection methods rather than explaining individual outlier observations by identifying their underlying causes. We use the explainability scores to evaluate which autoencoder based outlier detection methods are more explainable, rather than explaining individual outliers. Our explainability analysis and root cause identification serve different purposes and are orthogonal. Thus, they are incomparable.

Next, we proceed to introduce the two metrics that quantify the explainbility scores.

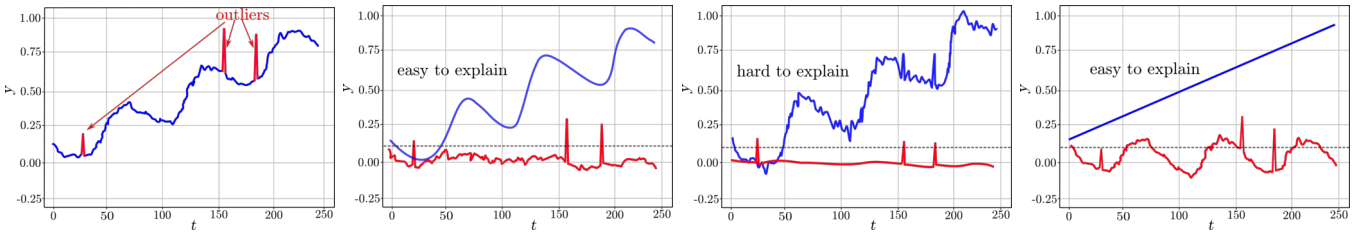
B. PRM-based Explainability Scores

We view a clean time series $\mathcal{T}_{\mathbf{L}} = \langle \mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_C \rangle$ as consisting of C training instances. We then employ a Linear Regression framework that considers $N+1$ independent variables $\{t^n\}_{n=0}^{N}$ to fit the training instances. When $N = 1$, we fit the training instances to a linear function to capture the linear trend of the clean time series; when $N > 1$, we fit the training instances using a high-order polynomial function that embodies both linear trends and nonlinear elements.

For a given N, we are able to obtain the best fitting polynomial function, denoted as $\mathcal{T}_{PRM}^{(N)}$. We then compute the root mean squared error $RMSE(\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{PRM}^{(N)})$ to measure the fit between $\mathcal{T}_{PRM}^{(N)}$ and the clean time series $\mathcal{T}_{\mathbf{L}}$. We define the explainability score $\mathcal{ES}_{PRM}$ as the smallest N such that the $RMSE$ is below a given threshold γ. In other words, given an *RMSE* threshold γ, we aim to find the smallest N such that the *RMSE* between the reconstructed, clean time series $\mathcal{T}_{\mathbf{L}}$ from the autoencoders and the time series $\mathcal{T}_{PRM}^{(N)}$ approximated by *PRM* with order N is below γ.

$$\mathcal{ES}_{PRM} = \min\{N \in \mathbb{N} | RMSE(\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{PRM}^{(N)}) < \gamma\} \tag{17}$$

Here, $\mathbb{N}$ is the natural numbers. For a specific γ, a smaller explainability score suggests higher explainability. A smaller



(a) Input time series $\mathcal{T}$ (b) Framework A, high explainability, high accuracy (c) Framework B, low explainability, high accuracy (d) Framework C, high explainability, low accuracy

Fig. 5: Intuition of explainability: in (b), (c), and (d), the blue curves show the clean time series $\mathcal{T}_{\mathbf{L}}^{A}$, $\mathcal{T}_{\mathbf{L}}^{B}$, and $\mathcal{T}_{\mathbf{L}}^{C}$, and the red curves show the outlier time series, obtained from frameworks A, B, and C, respectively. The dashed horizontal line represents the outlier score threshold 0.15. Frameworks A and B both find the three outliers as their outlier time series have three clear peaks which are above the horizontal line, while framework C is very inaccurate in terms of detecting outliers. Time series $\mathcal{T}_{\mathbf{L}}^{A}$ and $\mathcal{T}_{\mathbf{L}}^{C}$ can be explained well using trend and periodicity, thus having high explainability, while $\mathcal{T}_{\mathbf{L}}^{B}$ cannot be explained well because it includes many hard-to-explain variations, thus having low explainability. Framework A is highly desirable as it has both high accuracy and high explainability.

explainability score implies that it is possible to use a lower-order polynomial function, i.e., a simpler function, to fit the clean time series (within the same error threshold γ), which is easier for users to understand. The complexity of PRM is $O(C^2 \cdot D)$, where C is the time series length and D is the time series dimensionality.

C. SSA-based Explainability Scores

SSA [21] is an explainable time series analysis method that decomposes a time series into multiple components with different importance levels. These components indicate a *trend* (i.e., the most important component), multiple *periodicities* (i.e., less important components), and *noise* (i.e., the least important components). We apply SSA to a clean time series $\mathcal{T}_{\mathbf{L}}$ to obtain these components. We then construct time series $\mathcal{T}_{SSA}^{(N)}$ by combining the top-N most important components and apply $RMSE$ to measure the fit between $\mathcal{T}_{SSA}^{(N)}$ and $\mathcal{T}_{\mathbf{L}}$. We again define the explainability score $\mathcal{ES}_{SSA}$ as the smallest N such that the $RMSE$ is below a given threshold γ.

$$\mathcal{ES}_{SSA} = \min\{N \in \mathbb{N} | RMSE(\mathcal{T}_{\mathbf{L}}, \mathcal{T}_{SSA}^{(N)}) < \gamma\} \tag{18}$$

Given a specific γ, a lower explainability score indicates that it is possible to use fewer components to fit the clean time series, which indicates that the clean time series is easier to understand. The complexity of SSA is $O(D \cdot \min(B^3, K^3))$, where D is the time series dimensionality, B is the window size, and $K = C - B + 1$.

V. Experiments

A. Experimental Setup

Datasets: We use 7 time series datasets: (1) *GD*[1] contains two 20-dimensional time series and three 24-dimensional time series, which are collected from pick-and-place robots. Each time series contains from around 6,000 to 16,000 observations; (2) *HSS*[2] contains four 20-dimensional time series that are collected from a high storage system. Each time series contains from 19,000 to 25,000 observations; (3) *ECG*[3] contains 2-dimensional electrocardiogram time series collected from seven patients, each with 3,750 to 5,400 observations; (4) *NAB*[4] contains time series from six domains: urban traffic, temperature, CPU workload, Twitter posts, and exchange rates. Each domain has approximately 10 time series, each with 5,000 to 20,000 observations; (5) *S5*[5] includes a synthetic and a real-world dataset representing the workloads of different Yahoo services. Each dataset has around 100 time series, each with ca. 1,400 observations. (6) *2D*[3] contains seven sets of 2-dimensional time series that are converted from trajectories of hand writings. Each set has 3 time series, each with ca. 1,000 observations. (7) *SYN* contains 10 univariate time series with 2,000 observations that are generated from auto-regressive processes or basis functions such as sin and cosin. Then, we inject outliers into the generated time series. The outlier ratios (denoted as φ) of *GD*, *HSS*, *ECG*, *NAB*, *S5*, *2D*, and *SYN* are 0.8%, 16.7%, 4.9%, 9.8%, 0.9%, 39.2%, and 5%, respectively. All datasets contains both point and collective outliers. All datasets come with ground truth outlier labels. However, since we study unsupervised outlier detection, we do not use these labels during training. We only use them for testing, i.e., for evaluating accuracy. In addition, we train all methods using time series with outliers because datasets are typically not accompanied by clean time series without outliers for use in training. This setting enables us to study the robustness to outliers of different algorithms.

Baselines: We compare with 15 existing time series outlier detection approaches: (1) One-class Support Vector Machines (OCSVM) [34], a kernel based one-class classification method; (2) Local Outlier Factor (LOF) [12], a density based outlier detection method; (3) Isolation Forest (ISF) [35], an unsupervised tree-based outlier detection method; (4) Exponential Moving Average (EMA) [36], a time series smoothing method using weighted moving windows, where the weights for older observations decrease exponentially. (5) Seasonal-Trend decomposition using Local Regression (STL) [37], a time series smoothing method that decomposes time series

[1]https://kaggle.com/init-owl/genesis-demonstrator-data-for-machine-learning

[2]https://kaggle.com/init-owl/high-storage-system-data-for-energy-optimization

[3]https://cs.ucr.edu/∼eamonn/discords/

[4]https://github.com/numenta/nab/

[5]https://webscope.sandbox.yahoo.com/

into trends, seasonalities, and noise; (6) Singular Spectrum Analysis (SSA) [22], a time series decomposition method using Hankelize matrix decomposition; (7) Matrix Profile I (MP) [11], a state-of-the-art similarity based outlier detection method; (8) RandNet (RN) [38], an autoencoder ensemble for outlier detection. (9) CNN Autoencoder (CNNAE) [19], which treats time series as images and feeds them to a 2D CNN autoencoder to reconstruct them; (10) RNN Autoencoder (RNNAE) [19], [39], which reconstructs time series with a recurrent neural network (using LSTM units) based autoencoder; (11) BeatGAN (BGAN) [40], a generative model that forms an autoencoder via adversarial learning; (12) Donut (DONUT) [41], a variational autoencoder that reconstructs time series from stochastic latent spaces; (13) OmniAnomaly (OMNI) [42], a variational recurrent autoencoder that learns stochastic latent spaces for each observation of a time series; (14) Transformer Autoencoder (TAE) [43], which reconstructs time series by using an attention model. (15) Robust Deep Autoencoder (RDA) [44], which reconstructs time series by using a robust feed-forward autoencoder. Although LOF, OCSVM, ISF, and RN were originally proposed for non-time series data, they can be applied to time series with competitive accuracy [18]. Thus, we include them in the experiments.

Implementation Details: All methods are implemented by using Python 3.8. Further, PyTorch 1.1 is used for implementing all neural network based methods. Next, OCSVM, LOF, and ISF are implemented using Sklearn 1.19, and EMA, SSA, and MP are implemented using Numpy 1.15. Finally, STL is implemented using Statsmodels 0.12. All experiments are conducted on a Linux workstation with an AMD 64-core CPU with 512 GB RAM and 2 NVIDIA Titan V GPUs.

Hyperparameter Settings: Since we study unsupervised outlier detection, we are unable to tune the hyperparamters using labeled data. To ensure fair comparisons, we consider different hyperparameter settings, and report the *median* result for all methods. We do not report the best result because in unsupervised settings, we have no labeled data to enable identifying the best hyperparameters that lead to the best result. Specifically, we define a range for each hyperparameter. We then use random search with 200 random combinations to explore the hyperparameter space and identify a hyperparameter setting that gives the median result among all the explored hyperparameter settings. We then report this median result and consider this hyperparameter setting as the default setting. Next, we conduct experiments to study the sensitivity of different hyperparameters. To do so, each time, we vary a chosen hyperparameter in its range while fixing the other hyperparameters to their default settings. We proceed to provide the ranges for the hyperparameters.

For RAE and RDAE, we vary λ, λ_1, and λ_2 among 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, and 1; and we vary window size B among 10, 20, 50, 100, 200, and 400. For all the neural network based methods, we vary the number of layers, the kernel size (CNNAE, BGAN, RAE, and RDAE), and the number of attention heads (TAE) among 3, 5, 7, 9, and 11; next, we vary the number of kernels in each layer (CNNAE, BGAN, RAE, and RDAE), the number of hidden units (RN, RNNAE, DONUT, OMNI, and RDA), and the stochastic latent variable size (DONUT and OMNI) among 32, 64, 128, 256, 512, and 1024. For MP and EMA, we vary the pattern size among 5, 10, 20, 50, and 100. For RN and ISF, we vary the number of base models among 5, 10, 20, 50, 100, and 500. For LOF, we vary the number of neighbors among 5, 10, 20, 50, and 100. For STL, we vary the S (seasonal) and T (trend) coefficients among 1, 3, 5, 7, and 9. For OCSVM, we vary the kernel degree among 3, 5, 7, 9, and 11. We set ϵ to 10^{-5} (cf. Algorithms 1 and 2) as suggested in the literature [17], [27] that suggests $\epsilon \in [10^{-3}, 10^{-5}]$.

Evaluation Metrics: **Accuracy and Robustness:** We employ two metrics that consider all possible thresholds—Area Under the Curve of Precision-Recall (*PR*) [45] and Area Under the Curve of Receiver Operating Characteristic (*ROC*) [45]. *PR* and *ROC* can capture the performance of an outlier detection method without the need to choose any specific threshold. The higher *PR* and *ROC*, the more robust the method is.

Explainability: We use *RPM*-based and *SSA*-based post-hoc analysis to evaluate the explainability of different AE-based methods.

B. Experimental Results

Outlier Detection Accuracy: Tables II and III show that RAE and RDAE achieve on average the best and second best results, respectively. On some datasets, although our methods are not the best, they do not fall behind much. On datasets *HSS* and *2D*, our proposed methods are not the best. This is mainly because these datasets represents trajectories, i.e., time series of locations. More specifically, *HSS* is collected from a system that contains four short conveyor belts and two rails. These belts and rails are moving when the system is working, and the time series describe the positions of the belts and rails. Next, *2D* contains 2-dimensional time series that are converted from trajectories of hand writings. In such cases, distance-based and partition-based algorithms, such as LOF and ISF achieve better accuracy than the methods we propose because Euclidean space fits well with the data that represents physical locations. We leave improvements for this kind of data to future studies. RDAE performs the best on four out of seven datasets w.r.t. *PR* and *ROC*. Further, RDAE performs better than RAE in most the cases. This indicates that the dual AE architecture enhances the accuracy of a single AE. Because RDA cannot capture temporal dependencies, its performance is inferior to both RAE and RDAE.

Performance Improvement Significance: To determine whether the performance improvements of the proposed methods over the state-of-the-art methods are statistically significant, we conduct *t-tests* to assess the significance of the proposed methods against the baselines on the average results of all datasets. The *p-values* for both metrics are below 0.005. This indicates that the performance improvements over the state-of-the-art methods are statistically significant.

Effect of λ: We vary λ among 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, and 1. For RDAE we set $\lambda_1 = \lambda_2$ and vary them like λ. We also

TABLE II: Overall Accuracy, *PR*.

	OCSVM	LOF	ISF	EMA	STL	SSA	MP	RN	CNNAE	RNNAE	BGAN	DONUT	OMNI	TAE	RDA	RAE	RDAE
GD	0.112	0.079	0.087	0.081	0.078	0.134	0.094	0.112	0.116	0.128	0.109	0.097	0.131	0.088	0.113	0.141	**0.152**
HSS	0.197	0.227	0.217	0.158	0.156	0.167	0.174	0.146	0.207	**0.242**	0.214	0.203	0.219	0.195	0.168	0.197	0.223
ECG	0.121	0.112	0.119	0.089	0.125	0.103	0.116	0.105	0.107	0.118	0.103	0.131	0.144	0.138	0.115	0.127	**0.150**
NAB	**0.193**	0.145	0.167	0.136	0.149	0.148	0.173	0.168	0.177	0.163	0.151	0.188	0.165	0.175	0.162	0.159	0.190
S5	0.386	0.358	0.397	0.461	0.465	0.468	0.258	0.232	0.383	0.421	0.434	0.272	0.312	0.298	0.377	0.472	**0.481**
2D	0.468	0.476	0.511	0.482	0.435	0.422	0.398	0.426	0.401	0.413	0.408	0.414	0.427	0.405	0.415	0.498	**0.516**
SYN	0.102	0.121	0.126	0.135	0.129	0.128	0.113	0.105	0.116	0.125	0.144	0.132	0.118	0.124	0.127	**0.161**	0.158
Avg.	0.226	0.217	0.232	0.220	0.220	0.224	0.189	0.185	0.215	0.230	0.223	0.205	0.217	0.203	0.211	0.251	**0.267**

TABLE III: Overall Accuracy, *ROC*.

	OCSVM	LOF	ISF	EMA	STL	SSA	MP	RN	CNNAE	RNNAE	BGAN	DONUT	OMNI	TAE	RDA	RAE	RDAE
GD	0.582	0.562	0.651	0.528	0.552	0.544	0.545	0.731	0.641	0.608	0.673	0.664	0.658	0.652	0.613	0.744	**0.763**
HSS	0.523	0.553	**0.618**	0.521	0.537	0.514	0.528	0.526	0.560	0.537	0.549	0.525	0.557	0.563	0.550	0.565	0.581
ECG	0.526	0.563	0.557	0.542	0.537	0.519	0.534	0.524	0.574	0.552	0.547	0.531	0.551	0.542	0.556	0.577	**0.588**
NAB	0.533	0.519	0.551	0.521	0.528	0.548	0.516	0.532	0.541	0.524	0.524	0.563	0.557	0.536	0.528	0.567	**0.572**
S5	0.688	0.733	0.795	**0.835**	0.778	0.767	0.655	0.575	0.757	0.753	0.677	0.678	0.660	0.635	0.741	0.778	0.819
2D	0.541	**0.613**	0.598	0.528	0.501	0.528	0.507	0.523	0.529	0.551	0.568	0.573	0.588	0.571	0.575	0.611	0.609
SYN	0.549	0.563	0.547	0.592	0.598	0.584	0.565	0.561	0.603	0.594	0.562	0.564	0.569	0.571	0.573	0.608	**0.613**
Avg.	0.563	0.587	0.617	0.581	0.576	0.572	0.550	0.567	0.601	0.588	0.586	0.585	0.591	0.581	0.591	0.636	**0.649**

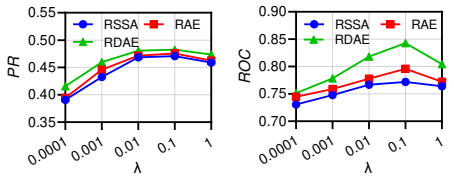
consider RSSA that employs RPCA to replace PCA as used in SSA. Intuitively, a small λ encourages more data to be kept in $\mathbf{S}$ and $\mathcal{T}_{\mathbf{S}}$ as outliers and forces only few observations to be considered as inliers. Instead, a large λ encourages more observations to be kept in $\mathbf{L}$ and $\mathcal{T}_{\mathbf{L}}$ as inliers; thus, fewer observations are considered as outliers. Due to the space limitation, we report on effect of λ on dataset *S5* only. The results on the other datasets show similar trends. Fig. 6 shows the results w.r.t. *PR* and *ROC*. All methods achieve the best performance when λ is between 10^{-2} and 10^{-1}. When $\lambda < 10^{-2}$, some clean data is mixed with both outliers in $\mathbf{S}$ and $\mathcal{T}_{\mathbf{S}}$, increasing the false positives. In contrast, when $\lambda > 10^{-1}$, only a few significant outliers are kept in $\mathbf{S}$ and $\mathcal{T}_{\mathbf{S}}$; thus, very few outliers are detected. Although this avoids false positives, some outliers are not detected, increasing the false negatives. When $\lambda \in (10^{-2}, 10^{-1})$, the best trade-off is achieved.

Effect of B: We study the effect of the window size B in the frameworks SSA and RDAE that both employ lagged matrices created based on B. We also consider RSSA that employs RPCA to replace PCA as used in SSA. We vary B among 10, 20, 50, 100, 200, and 400. We report findings on dataset *S5* only. The results on the other datasets show similar trends. Fig. 7 shows the results w.r.t. *PR* and *ROC*. All methods achieve the peak performance when $B = 200$, which suggests that $B = 200$ is the most appropriate setting for all the datasets. This observation is consistent with the recommendation by Khan and Poskitt [46] that B is set to $(\ln C)^{\psi}$, where $\psi \in (1.5, 3.0)$ and C is the length of a time series (see Section II). In *S5*, C is around 1,400.

Ablation Study: We study the effect of each component in RDAE by removing $f_{1_{\theta_1}}(\cdot)$ (denoted as RDAE-f1), removing $f_{2_{\theta_2}}(\cdot)$ (denoted as RDAE-f2), and removing both (denoted as RDAE-f1f2). RDAE-f1f2 can be viewed as RDA [44]. We also consider RSSA that employs RPCA to replace PCA as used in SSA. We report findings on dataset *S5* only. The results on the other datasets show similar trends. Fig. 8 shows the results w.r.t. *PR* and *ROC*. RDAE outperforms RAE that only uses the time series view, suggesting that multi-view (i.e., both time series and matrix representation) is superior to single-view (i.e., only time series representation). RDAE outperforms RDAE-f1f2 that only uses the (single-view) lagged matrix. RDAE-f1 and RDAE-f2 outperform RDAE-f1f2 w.r.t. *PR* and *ROC*. Further, RDAE-f1 outperforms RDAE-f2 w.r.t. *PR* and *ROC*, which suggests that the outer AE $f_{2_{\theta_2}}(\cdot)$ is more important than $f_{1_{\theta_1}}(\cdot)$ because that is only a non-linear transformation that offers input for the inner AE, while $f_{2_{\theta_2}}(\cdot)$ performs the reconstruction of the time series view. RDAE achieves the best performance, which suggests that the two AEs together with the additional nonlinear transformation offered by $f_{1_{\theta_1}}(\cdot)$ yields the most powerful framework. Further, we consider RDAE+MA, where we use a simple moving average to smooth the input time series $\mathcal{T}$. The smoothed time series is then transformed to the smoothed lagged matrix $\hat{\mathbf{M}}$. The results show that RDAE+MA is worse than RDAE, suggesting that our proposed non-linear transformation is better than moving average smoothing.

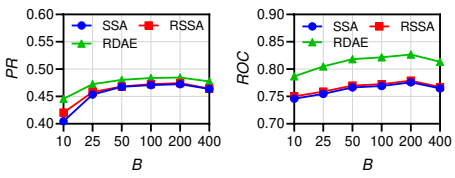
Robustness: We study the robustness of the proposed methods by removing the robustness considerations of RAE and RDAE to create two non-robust counterparts, denoted as N-RAE and N-RDAE, respectively. Specifically, for N-RAE, we employ an 1D CNN-based AE to reconstruct time series $\mathcal{T}$. The reconstructed time series is considered as the clean time series $\mathcal{T}_{\mathbf{L}}$. Next, the outlier scores are computed by differencing $\mathcal{T}$ and $\mathcal{T}_{\mathbf{L}}$. For N-RDAE, we employ a 2D CNN-based AE as the inner AE to reconstruct the lagged matrix $\hat{\mathbf{M}}$. The reconstruction is considered as the clean lagged matrix $\mathbf{L}$. Then, we employ an 1D CNN-based AE as the outer AE to reconstruct the time series, which is obtained from $\mathbf{L}$. The reconstructed time series is considered as the clean time series $\mathcal{T}_{\mathbf{L}}$. As before, the outlier scores are computed by differencing $\mathcal{T}$ and $\mathcal{T}_{\mathbf{L}}$. Fig. 9 shows the experimental results of RAE vs. N-RAE and RDAE vs. N-RDAE w.r.t. *PR* and *ROC*. We report results on dataset *S5* only. The results on the other datasets show similar trends. The results show that RAE outperforms N-RAE and that RDAE outperforms N-RDAE, justifying the robustness design choices used in RAE and RDAE. Further, we observe that many time series in experimental datasets contain only few outliers but these outliers can affect the non-robust AEs, thus making N-RAE and N-RDAE work poorly.

Effect of ϵ: We study the effect of ϵ in RAE and RDAE. Specifically, we vary ϵ among 10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, and 1. We report findings on dataset *S5* only. The results on the other datasets exhibit similar trends. Fig. 10



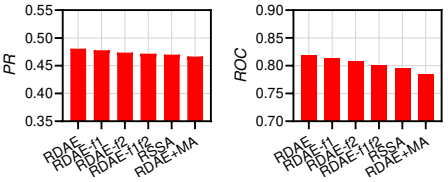
(a) *PR*. (b) *ROC*.

Fig. 6: Effect of λ.



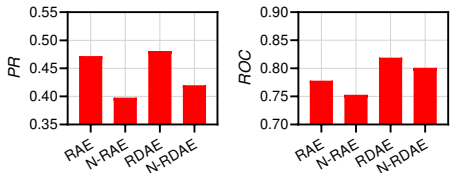
(a) *PR*. (b) *ROC*.

Fig. 7: Effect of B.



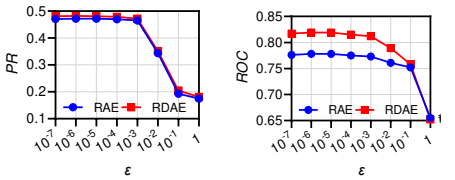
(a) *PR*. (b) *ROC*.

Fig. 8: Ablation Study.



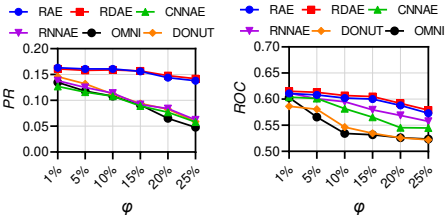
(a) *PR*. (b) *ROC*.

Fig. 9: Robustness.



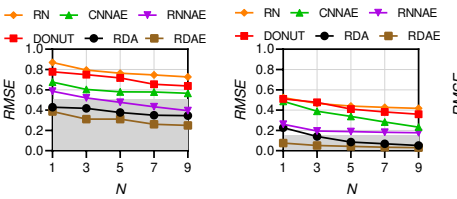
(a) *PR*. (b) *ROC*.

Fig. 10: Effect of ϵ.



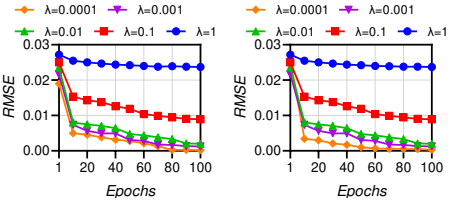
(a) *PR*. (b) *ROC*.

Fig. 11: Effect of φ.

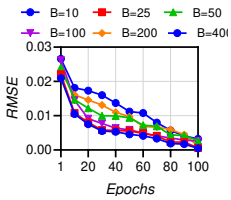


(a) *PHE-PRM*. (b) *PHE-SSA*.

Fig. 12: Post-hoc Explainability.



(a) λ, RAE. (b) λ, RDAE.



(c) B, RDAE.

Fig. 13: Effect of λ and B, Convergence Analysis.

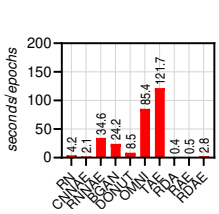


Fig. 14: Runtimes.

shows the results w.r.t. *PR* and *ROC*. We observe that when $\epsilon \in [10^{-3}, 10^{-5}]$, the accuracy only changes insignificantly. When $\epsilon > 10^{-3}$, the accuracy drops because the training has not converged yet. When $\epsilon < 10^{-5}$, the accuracy does not change much but the training takes a very long time. Thus, our choice $\epsilon = 10^{-5}$ is consistent with the literature [17], [27].

Effect of Outlier Ratios: We study the effect of the outlier ratios φ of a dataset. Specifically, we vary φ on *SYN* among 1%, 5%, 10%, 15%, 20%, and 25%. We include CNNAE, RNNAE, DONUT, and OMNI as baselines due to their competitive accuracy (cf. Tables II and III). Fig. 11 shows the results w.r.t. *PR* and *ROC*. We see that RAE and RDAE can maintain their accuracy while the accuracy of the baselines drop quickly. This suggests that our proposed methods offer added robustness to contaminated data.

Explainability: We conduct a post-hoc explainability analysis to gain insight into the explainability of the AE-based methods: CNNAE, RNNAE, RN, DONUT, RAE, and RDAE. For RAE and RDAE, we consider $\mathcal{T}_{\mathbf{L}}$ as the clean time series. For CNNAE, RNNAE, and DONUT, we consider the reconstructed time series as the clean time series. For RN, we use the average of all reconstructed time series from the ensemble as the clean time series. Given a clean time series, we employ *PHE-PRM* and *PHE-SSA* (cf. Section IV) to quantify the explainability of the frameworks when varying N among 1, 3, 5, 7, and 9. Recall the *RMSE* threshold γ in Eqs. 17 and 18. We aim to identify the smallest N such that the *RMSE* is at most γ. In other words, with a specific threshold γ, the model with the smaller N that can produce an *RMSE* that is at most γ is more explainable. Fig. 12 shows the *PHE-PRM* and *PHE-SSA* results on the *S5* dataset. No matter which γ we choose, RAE and RDAE have the smallest (and thus the best) explainability scores. For example, for *PHE-PRM* and with $\gamma = 0.5$, RAE and RDAE both have explainability score 1, meaning that a linear function can approximate the clean time series within *RMSE* 0.5. In contrast, RNNAE needs to use a polynomial function of degree 5. Further, CNNAE, DONUT, and RN fail to achieve an *RMSE* below 0.5 when using up to degree 9 polynomial functions. The results show that the proposed methods, which take into account robustness, excel at learning fundamental patterns of time series and thus are easier to explain. This indicates that RAE and RDAE are the most explainable and are able to learn fundamental patters of time series. The observation also holds when using *PHE-SSA*. When choosing $\gamma = 0.15$, RDAE has explainability score 1, indicating that RDAE is able to produce a clean time series that can be explained by a linear trend (i.e., the most important SSA component). RAE is the second best with an explainability score of 3. The other methods are not explainable by up to 9 SSA components when $\gamma = 0.15$.

We can also consider explainability from the different perspective, where we fix N. We observe that RDAE and RAE always have the lowest *RMSE* values. This suggest that when using a specific post-hoc analysis model (i.e., a fixed N) to fit the clean time series, the post-hoc analysis model best fits the clean time series derived from the RDAE and RAE. This again suggests that RAE and RDAE have the best explainability. Next, we consider the concrete example in Fig. 1. The key features of the clean time series obtained by RDAE, e.g., the clear periodic pattern, is easier for a human expert to understand what observations should occur at different timestamps. In contrast,

the clean time series obtained by RNNAE includes many hard-to-understand variations. The above observation is justified by a post-hoc analysis–when using *PHE-SSA* with $\gamma = 0.1$, we get $N = 2$ for RDAE and $N = 9$ for RNNAE, suggesting that RDAE has high explainability. We also observe that RDAE achieves higher accuracy by having a more explainable clean time series. See the highlighted parts in Fig. 1 that include two outliers. RDAE gives two high outlier scores based on its clean time series, making it easy to identify them as outliers, whereas RNNAE gives them small outlier scores. This suggests that a more explainable clean time series also contributes improved accuracy.

Empirical Convergence Analysis: We study the convergence for (i) RAE with hyperparameters λ and (ii) RDAE with with hyperparameter λ and B. The convergence analysis is conducted by measuring the difference (i.e., *root mean square error* (*RMSE*)) between the original time series $\mathcal{T}$ and the clean time series $\mathcal{T}_{\mathbf{L}}$. To evaluate the effect of λ, we vary λ among 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, and 1 while keeping $B = 50$. To evaluate the effect of B, we vary B among 10, 20, 50, 100, 200, and 400 while keeping $\lambda = 10^{-1}$. We show the results of the convergence analysis for two random time series from dataset *S5* only, due to the space limitation. The convergences for the other time series exhibit similar trends. Fig. 13 shows the convergence for RAE and RDAE. Both RAE and RDAE converge quickly with small λ during the first 10 epochs. Moreover, the convergences are insensitive to B but are sensitive to λ, and they converge in all cases.

Runtimes: We first evaluate the training times of the neural network based methods RN, CNNAE, RNNAE, BGAN, DONUT, OMNI, TAE, RDA, RAE, and RDAE. We report findings on dataset *S5* only. The results on the other datasets show similar trends. Fig. 14 shows the training *runtime* (*seconds/epoch*). Our methods run extremely fast because they do not perform recursive computations. RDA and RAE are the fastest methods. RDAE has a very competitive runtime and is only slightly slower than CNNAE. The other methods run considerably slower due to several reasons: (i) some perform recursive computations (e.g., RNNAE and OMNI); (ii) some are ensemble models that require training of multiple base models (e.g., RN); (iii) some have sampling tricks that take long time (e.g., DONUT and OMNI); (iv) some have complicated objective functions that do not converge easily (e.g., BGAN with the minimax objective function); and (v) some have expensive attention score computations (e.g., TAE). Next, the testing runtimes of the two proposed frameworks and all the other methods are small, i.e., less than 0.1 *seconds*, making them applicable to online outlier detection in streaming settings. This evaluation offers evidence that the proposed RAE and RDAE methods can work in time-critical settings while obtaining good accuracy.

VI. Related Work

Outlier detection. AEs with 1D CNNs and RNNs have been used for temporal outlier detection [19], [47]. AEs are combined with adversarial training and variational inference for time series anomaly detection [48], [49]. AEs are also applied in denoising [50]. Compared with an existing robust approach [44], the proposed frameworks can handle time series, take into account temporal information by using a lagged matrix thus supporting multi-view representations, and support explainability. In addition, unlike existing studies that use a single AE, our RDAE framework combines two AEs that help each other identify outliers using two representations of time series data, which improves accuracy. RDAE is also different from AE ensembles, where RDAE is a single model that consists of two AEs but an AE ensemble includes many basic models where each basic model is an AE [38], [51].

RPCA has been employed to identify outliers [52]. However, no existing studies apply RPCA to time series. Our frameworks build on RPCA, and the frameworks work for time series.

Explainable Machine Learning. Explainability is categorized as either intrinsic or post-hoc [16]. Most existing explainable methods are proposed for supervised problems [53]. Only few explainable methods exist for outlier detection, and they often rely on human experts annotation [11], rendering them supervised proposals. These explainable methods focus on indicating the root causes of outliers. Zhang *et al*. [32] propose a framework to explain outlier events in time series based on annotated events. Recently, Rad *et al*. [33] propose an unsupervised root cause analysis method to explain outliers by indicating the most anomalous dimension in high-dimensional time series. In contrast, we quantify the explainability of autoencoder based outlier detection methods rather than explaining individual outlier observations and what causes an outlier observation. Our explainability analysis method is used to evaluate which autoencoder based outlier detection methods are more explainable, rather than explaining individual outliers.

VII. Conclusion

We propose two explainable and robust autoencoder frameworks for unsupervised time series outlier detection. The frameworks represent the first attempt to improve two key aspects of existing neural net based autoencoders: low explainability and high vulnerability to outliers. The frameworks decompose a time series into a clean and an outlier time series, which provides theoretical underpinnings and makes them robust to outliers. We provide a post-hoc explainability analysis method to quantify model explainability. Experimental studies show that the frameworks are effective and outperform strong baselines and state-of-the-art methods. In future research, it is of interest to study outlier detection under slightly relaxed settings that are other than fully unsupervised settings, e.g., weakly supervised settings [54]. It is also of interest to study different means to further improve outlier detection accuracy, e.g., using ensemble learning [51] and curriculum learning [55], and considering time series of location related information.

Acknowledgments

This work was supported in part by Independent Research Fund Denmark under agreements 8022-00246B and 8048-00038B, the VILLUM FONDEN under agreements 34328 and 40567, Huawei Cloud Database Innovation Lab, and the Innovation Fund Denmark center, DIREC.

REFERENCES

[1] R.-G. Cirstea, T. Kieu, C. Guo, B. Yang, and S. J. Pan, "EnhanceNet: Plugin neural networks for enhancing correlated time series forecasting." in *ICDE*, 2021, pp. 1739–1750.

[2] R.-G. Cirstea, B. Yang, C. Guo, T. Kieu, and S. Pan, "Towards spatio-temporal aware traffic time series forecasting," in *ICDE*, 2022, to appear.

[3] J. Hu, B. Yang, C. Guo, C. S. Jensen, and H. Xiong, "Stochastic origin-destination matrix forecasting using dual-stage graph convolutional, recurrent neural networks," in *ICDE*, 2020, pp. 1417–1428.

[4] X. Wu, D. Zhang, C. Guo, C. He, B. Yang, and C. S. Jensen, "AutoCTS: Automated correlated time series forecasting," *PVLDB*, vol. 15, no. 4, pp. 971–983, 2022.

[5] R.-G. Cirstea, B. Yang, and C. Guo, "Graph attention recurrent neural networks for correlated time series forecasting." in *MileTS19@KDD*, 2019.

[6] S. A. Pedersen, B. Yang, and C. S. Jensen, "Anytime stochastic routing with hybrid learning," *Proc. VLDB Endow.*, vol. 13, no. 9, pp. 1555–1567, 2020.

[7] S. B. Yang, C. Guo, and B. Yang, "Context-aware path ranking in road networks," *IEEE Trans. Knowl. Data Eng.*, 2020, to appear.

[8] C. Guo, B. Yang, J. Hu, C. S. Jensen, and L. Chen, "Context-aware, preference-based vehicle routing," *VLDB J.*, vol. 29, no. 5, pp. 1149–1170, 2020.

[9] S. A. Pedersen, B. Yang, and C. S. Jensen, "Fast stochastic routing under time-varying uncertainty," *VLDB J.*, vol. 29, no. 4, pp. 819–839, 2020.

[10] H. Liu, C. Jin, B. Yang, and A. Zhou, "Finding top-k optimal sequenced routes," in *ICDE*, 2018, pp. 569–580.

[11] C. M. Yeh, Y. Zhu, L. Ulanova, N. Begum, Y. Ding, H. A. Dau, D. F. Silva, A. Mueen, and E. J. Keogh, "Matrix profile I: all pairs similarity joins for time series: A unifying view that includes motifs, discords and shapelets," in *ICDM*, 2016, pp. 1317–1322.

[12] M. M. Breunig, H. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *SIGMOD*, 2000, pp. 93–104.

[13] G. Hinton and R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.

[14] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Distinguishing trajectories from different drivers using incompletely labeled trajectories," in *CIKM*, 2018, pp. 863–872.

[15] N. Tishby, F. C. Pereira, and W. Bialek, "The information bottleneck method," *arXiv preprint physics/0004057*, 2000.

[16] M. Du, N. Liu, and X. Hu, "Techniques for interpretable machine learning," *Commun. ACM*, vol. 63, no. 1, pp. 68–77, 2020.

[17] E. J. Candès, X. Li, Y. Ma, and J. Wright, "Robust principal component analysis?" *J. ACM*, vol. 58, no. 3, pp. 11:1–11:37, 2011.

[18] C. C. Aggarwal, *Outlier Analysis*. Springer, 2013.

[19] T. Kieu, B. Yang, and C. S. Jensen, "Outlier detection for multidimensional time series using deep neural networks," in *MDM*, 2018, pp. 125–134.

[20] W. Wang, R. Arora, K. Livescu, and J. A. Bilmes, "On deep multi-view representation learning," in *ICML*, 2015, pp. 1083–1092.

[21] A. Groth and M. Ghil, "Multivariate singular spectrum analysis and the road to phase synchronization," *Phys. Rev. E*, vol. 84, no. 3, pp. 9528–9544, 2011.

[22] N. Golyandina, V. Nekrutkin, and A. A. Zhigljavsky, *Analysis of Time Series Structure - SSA and Related Techniques*. Chapman & Hall/CRC Press, 2001.

[23] N. H. Packard, J. P. Crutchfield, J. D. Farmer, and R. S. Shaw, "Geometry from a time series," *Phys. Rev. Lett.*, vol. 45, no. 9, p. 712, 1980.

[24] Y. Xia, X. Cao, F. Wen, G. Hua, and J. Sun, "Learning discriminative reconstructions for unsupervised outlier removal," in *ICCV*, 2015, pp. 1511–1519.

[25] E. J. Candes and T. Tao, "Decoding by linear programming," *IEEE Trans. Inf. Theory*, vol. 51, no. 12, pp. 4203–4215, 2005.

[26] D. Ge, X. Jiang, and Y. Ye, "A note on the complexity of l_p minimization," *Math Program*, vol. 129, no. 2, pp. 285–299, 2011.

[27] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine learning*, vol. 3, no. 1, 2011.

[28] N. Parikh and S. Boyd, "Proximal algorithms," *Foundations and Trends® in Optimization*, vol. 1, no. 3, 2014.

[29] R. Nishihara, L. Lessard, B. Recht, A. Packard, and M. I. Jordan, "A general analysis of the convergence of ADMM," in *ICML*, 2015, pp. 343–352.

[30] M. Hong, Z. Luo, and M. Razaviyayn, "Convergence analysis of alternating direction method of multipliers for a family of nonconvex problems," *SIAM J. Optim.*, vol. 26, no. 1, pp. 337–364, 2016.

[31] T. Hastie, J. H. Friedman, and R. Tibshirani, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, ser. Springer Series in Statistics. Springer, 2001.

[32] H. Zhang, Y. Diao, and A. Meliou, "Exstream: Explaining anomalies in event stream monitoring," in *EDBT*, 2017, pp. 156–167.

[33] B. Rad, F. Song, V. Jacob, and Y. Diao, "Explainable anomaly detection on high-dimensional time series data," in *DEBS*, 2021, pp. 2–14.

[34] L. M. Manevitz and M. Yousef, "One-class SVMs for document classification," *J. Mach. Learn. Res.*, vol. 2, pp. 139–154, 2001.

[35] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *ICDM*, 2008, pp. 413–422.

[36] R. G. Brown, *Smoothing, Forecasting and Prediction of Discrete Time Series*, ser. Dover Phoenix Editions. Dover Publications, 1963.

[37] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "Stl: A seasonal-trend decomposition procedure based on loess," *J. Off. Stat.*, vol. 6, no. 1, pp. 3–73, 1990.

[38] J. Chen, S. Sathe, C. C. Aggarwal, and D. S. Turaga, "Outlier detection with autoencoder ensembles," in *SDM*, 2017, pp. 90–98.

[39] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," *CoRR*, vol. abs/1607.00148, 2016.

[40] B. Zhou, S. Liu, B. Hooi, X. Cheng, and J. Ye, "BeatGAN: Anomalous rhythm detection using adversarially generated time series," in *IJCAI*, 2019, pp. 4433–4439.

[41] H. Xu, W. Chen, N. Zhao, Z. Li, J. Bu, Z. Li, Y. Liu, Y. Zhao, D. Pei, Y. Feng, J. Chen, Z. Wang, and H. Qiao, "Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications," in *WWW*, 2018, pp. 187–196.

[42] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *SIGKDD*, 2019, pp. 2828–2837.

[43] H. Meng, Y. Zhang, Y. Li, and H. Zhao, "Spacecraft anomaly detection via transformer reconstruction error," in *ICASSE*, 2020, pp. 351–362.

[44] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *SIGKDD*, 2017, pp. 665–674.

[45] C. Sammut and G. I. Webb, *Encyclopedia of Machine Learning*. Springer, 2017.

[46] A. R. Khan and D. S. Poskitt, "Window length selection and signal-noise separation and reconstruction in singular spectrum analysis," *Monash Econometrics and Business Statistics*, vol. 23, no. 11, pp. 23–42, 2011.

[47] D. Campos, T. Kieu, C. Guo, F. Huang, K. Zheng, B. Yang, and C. S. Jensen, "Unsupervised time series outlier detection with diversity-driven convolutional ensembles," *PVLDB*, vol. 15, no. 3, pp. 611–623, 2022.

[48] X. Chen, L. Deng, F. Huang, C. Zhang, Z. Zhang, Y. Zhao, and K. Zheng, "Daemon: Unsupervised anomaly detection and interpretation for multivariate time series," in *ICDE*, 2021, pp. 2225–2230.

[49] T. Kieu, B. Yang, C. Guo, R.-G. Cirstea, Y. Zhao, Y. Song, and C. S. Jensen, "Anomaly detection in time series with robust variational quasi-recurrent autoencoders," in *ICDE*, 2022, to appear.

[50] J. Xie, L. Xu, and E. Chen, "Image denoising and inpainting with deep neural networks," in *NIPS*, 2012, pp. 350–358.

[51] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Outlier detection for time series with recurrent autoencoder ensembles," in *IJCAI*, 2019, pp. 2725–2732.

[52] H. Xu, C. Caramanis, and S. Sanghavi, "Robust principal component analysis via outlier pursuit," in *NIPS*, 2010, pp. 2496–2504.

[53] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *ICML*, vol. 70, 2017, pp. 1885–1894.

[54] S. B. Yang, C. Guo, J. Hu, B. Yang, J. Tang, and C. S. Jensen, "Temporal path representation learning with weakly-supervised contrastive curriculum learning," in *ICDE*, 2022.

[55] S. B. Yang, C. Guo, J. Hu, J. Tang, and B. Yang, "Unsupervised path representation learning with curriculum negative sampling," in *IJCAI*, 2021, pp. 3286–3292.